ARQUITETURA DE SOFTWARE & INTELIGÊNCIA ARTIFICIAL

# GastoCerto IA

Agente Pessoal de Gestão Financeira com API Gemini, Engenharia de Prompts, DDD e Frontend Orientado a Features

Mario Ferreira | Maicon Cruz

PROPOSTA DO SISTEMA

# O que é o GastoCerto IA?

### Linguagem Natural

Registro conversacional de despesas diárias.

### Classificação IA

Categorização automática via Google Gemini.

### Controle de Metas

Monitoramento de teto mensal em tempo real.

EXPERIÊNCIA DO USUÁRIO

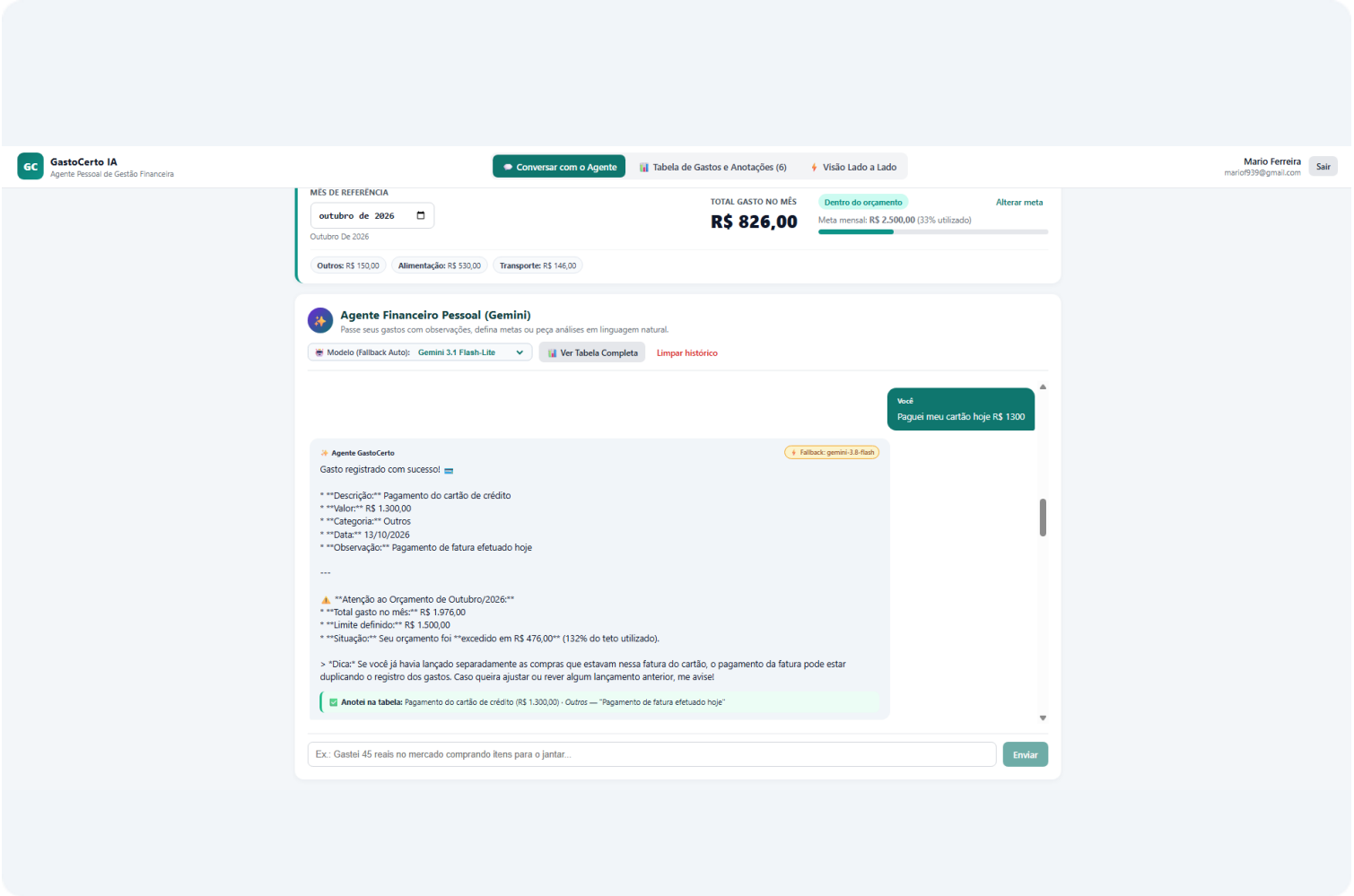
# Agente Conversacional Integrado

O usuário envia uma frase simples e o agente interpreta, persiste o lançamento e analisa o orçamento.

**Extração:** Valor, data, categoria e anotação.

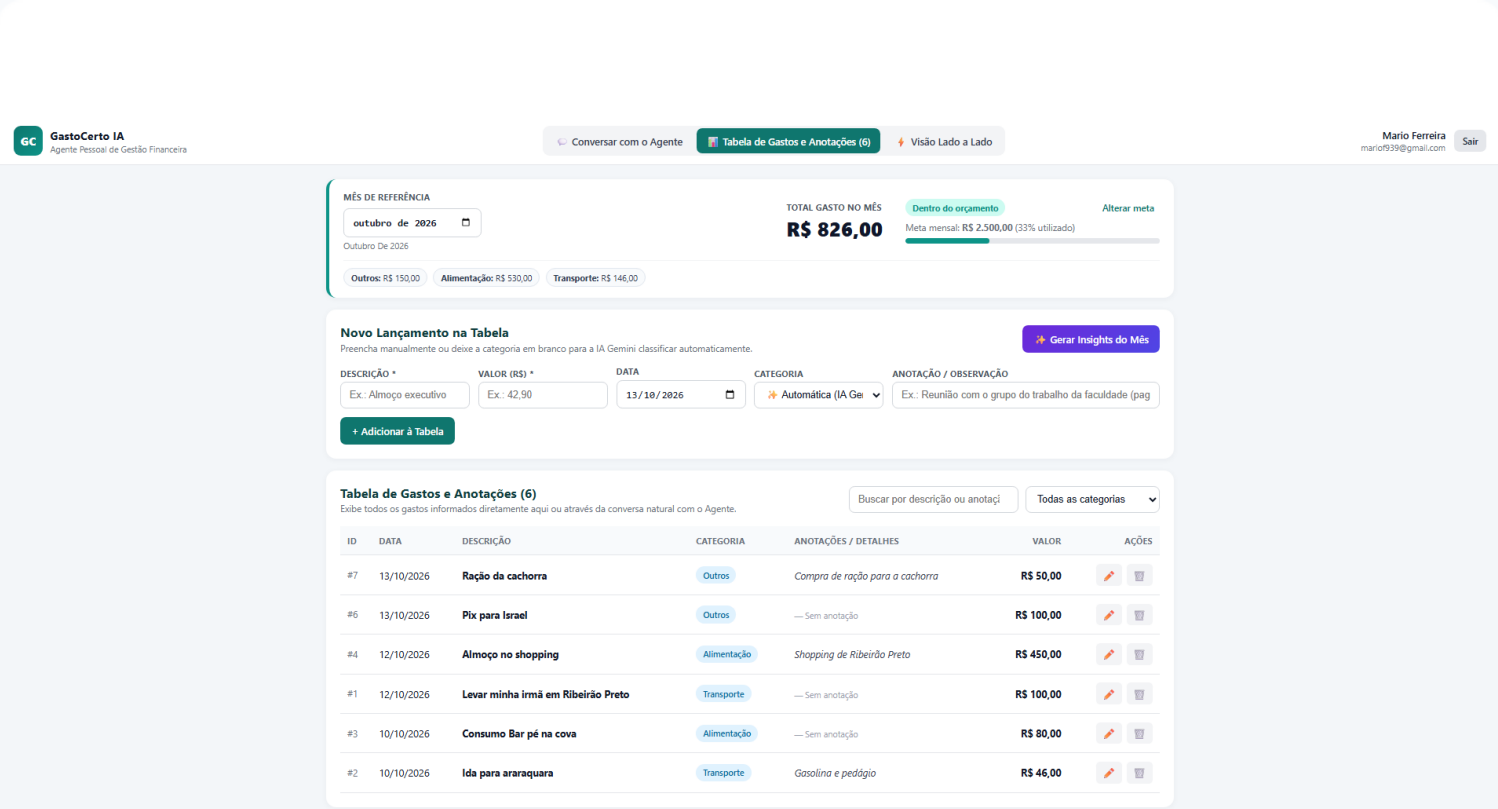
**Alerta de Teto:** Aviso imediato de orçamento excedido.

**Dica Proativa:** Prevenção de duplicidade na fatura.



VISÃO ESTRUTURADA

# Tabela de Gastos e Insights




Sincronia total entre os lançamentos via chat e o gerenciamento manual em tabela.

**Categoria Automática:** IA classifica quando deixada em branco.

**Insights do Mês:** Botão dedicado para análises financeiras.

**Visão Híbrida:** Modos Chat, Tabela ou Lado a Lado.

# Integração com Economia Awesome API

# Entrada e retorno da API de Cotação

**Entrada da API (Através da Rota)**

**https://economia.awesomeapi.com.br/last/USD-BRL**

**Saída da API**


```
{
  "USDBRL": {
    "code": "USD",
    "codein": "BRL",
    "name": "Dólar Americano/Real Brasileiro",
    "high": "5.4406",
    "low": "5.4369",
    "varBid": "-0.0028",
    "pctChange": "-0.051476",
    "bid": "5.4369",
    "ask": "5.4383",
    "timestamp": "1793627745",
    "create_date": "2026-11-02 10:55:45"
  }
}
```

PILAR 01 • INTELIGÊNCIA ARTIFICIAL

# Integração com API Gemini & Prompt Engineering

INTEGRAÇÃO COM A API DO GEMINI

# Resiliência e Múltiplas Capacidades

### Fallback Automático de Modelos

Alternância transparente entre modelos Flash-Lite e Flash.

### Extração de Dados Estruturados

Conversão de texto livre em payload validado.

### Inferência de Categoria

Classificação automática no formulário manual de gastos.

### Consultoria Contextual

Geração de insights mensais sobre hábitos de consumo.

PROMPT ENGINEERING

# Pipeline de Construção do Prompt

01

**System Persona**

Papel de consultor financeiro objetivo.

02

**Injeção de Contexto**

Data atual, meta mensal e total acumulado.

03

**Regras de Domínio**

Categorias válidas e cálculo de teto.

04

**Output Estruturado**

Resposta formatada mais ação de registro.

ENGENHARIA DE PROMPTS NA PRÁTICA

# Da Frase Natural à Ação no Sistema

### Entrada + Estado Injetado

**Mensagem:** "Paguei meu cartão hoje R$ 1300"

**Contexto Temporal:** 13/10/2026 (Outubro/2026)

**Estado do Orçamento:** Limite R$ 1.500 | Prévio R$ 676

### Saída Determinística do Agente

**Persistência:** R$ 1.300,00 em *Outros* (13/10/2026)

**Cálculo de Alerta:** Total R$ 1.976 (132% utilizado)

**Heurística:** Alerta sobre duplicidade de compras do cartão

PILAR 02 • ARQUITETURA BACKEND

# Organização com Domain-Driven Design (DDD)

| Camada DDD | Elementos Principais | Responsabilidade no Projeto |
|---|---|---|
| **Domain** | Entidades, Value Objects, Contratos | Regras de gastos, categorias e metas mensais |
| **Application** | Use Cases, DTOs, Orquestrador IA | Fluxo do chat, montagem de contexto e comandos |
| **Infrastructure** | Gemini Client, Repositórios, Banco | Integração externa com SDK Gemini e persistência |
| **API** | Controllers, Rotas, Middlewares | Exposição REST segura para o frontend |

DESACOPLAMENTO E MANUTENIBILIDADE

# Por que DDD em um Sistema com IA?

- **Domínio Protegido:** A regra financeira não depende diretamente da SDK do Google Gemini.
- **Estratégia de Fallback Isolada:** A troca entre modelos ocorre na infraestrutura sem afetar os casos de uso.
- **Alta Testabilidade:** Regras de orçamento e prompts podem ser testados com mocks de forma determinística.
- **Linguagem Ubíqua:** Termos como *Gasto*, *Meta Mensal* e *Categoria* guiam código e prompts.

PILAR 03 • ARQUITETURA FRONTEND

# Frontend Orientado a Features

```
src/
 ├── app/
 ├── presentation/
 |     ├── agent-chat/
 |     ├── expenses-table/
 |     ├── budget-summary/
 |     └── auth/
 └── shared/
       ├── components/
       └── services/
```

Código agrupado por funcionalidade de negócio em vez de pastas técnicas genéricas.

**Alta Coesão:** Componentes, hooks e serviços juntos por domínio.

**Composição Flexível:** Facilita a "Visão Lado a Lado" unindo Chat e Tabela.

**Escalabilidade:** Novas features não poluem módulos existentes.

# Obrigado!